Remote Yoga

Justin Stanley, Stephanie Seward, and Austin Doolittle

Abstract

Edge computing and IoT devices has enabled the modern athletic and health monitoring market to blossom, creating new products that yield important insights into the biometrics of their users. While these advances are useful and interesting, they lack the ability to provide real-time feedback to their users on how to improve their form while accomplishing some physical task. We propose YogaPose, a distributed infrastructure of yoga instructors and students. Our solution enables real-time feedback by leveraging CNN architectures trained for Yoga pose recognition and human pose estimation.

Introduction

The advent of the Internet of Things and Edge computing have enabled many industries in the marketplace to provide new solutions to age old problems. These advances in technology have made significant impact on the fitness industry with wearable biometric sensors that provide real-time insights into the vitals of athlete. New hardware with integrated sensor technology has also enabled in-home personal training, and direct communication channels with health professionals are now within reach of the average consumer. Technology once reserved for elite and professional athletes is accessible.

Most of these advances have focused on fitness tracking in the form of biometrics. Applications focused on movement tracking and form have yet to hit the market at scale leveraging edge devices. The team wanted to focus on the world of athletics, but we sought inspiration from the world of videogames and movement trackers like PlaystationMove. We propose a novel application that enables fitness professionals to not only administer fitness classes, but also allows those professionals to view progress and to provide feedback backed by big data analytics. This system is also inexpensive, and easily implemented using preexisting cloud infrastructure and embedded devices. We've developed YogaPose, pose estimation software that allows pose instructors to teach class at scale and analyze which of their students follow along successfully with recommended courses of instruction. It's yoga class with an instructor available anytime, anywhere, who can give instantaneous feedback, even without an internet connection, and feedback based on student trends when students are connected.

Background

A number of different biometric monitoring services exist in the marketplace today. Fitbit[10] was an early contender by providing small, bluetooth enabled devices which monitor the vitals of the individual wearer. The measured biometrics are then made available to the user, however the user is responsible for interpreting and acting on these metrics. Apple has also contributed to this market with their Apple Watch devices[3]. These devices provide professional level heartbeat and blood oxygen level monitoring, but, like Fitbit, rely on the user to interpret and react to the displayed metrics.

The smart home workout equipment market is much younger than wearable biometric devices, but that does not stop it from rapidly growing. Arguably the most recognizable name in this market is Peloton[5], which is a luxury workout bike that connects athletes to professionally guided spin classes. Instructors have the ability to monitor the live statistics of athlete performance, including current distance and speed. Another example of professionally guided in-home workout equipment is the company Tonal[9]. Tonal provides a digital resistance training device that creates and displays various workouts on a large display. It tracks the progress of the weight that the user is able to lift, and also provides detailed instructional videos on how to perform different workouts. Tonal, however, does not provide any professional human feedback on form or performance.

While professional human analysis is beneficial to workout regiments, automation opportunities for progress tracking abound. Websites like TrainingPeaks[2] automatically tracks athlete progress, but requires athletes to interact with the platform and upload their own data to receive performance tracking services. Each of these services is oriented towards tracking biometrics, rather than movement, potentially increasing injury risk for athletes who fail to achieve good form and don't have coaches present.

Convolutional neural networks (CNNs), first introduced with AlexNet in 2014[6], have made great strides in processing image data. AlexNet was constructed to perform the task of image classification, however many more problems were able to be solved. Recent studies have shown that CNNs are quite good at the task of human pose estimation[4,8]. We intend to leverage this technology in our proposed architecture to give yoga students near real time feedback about whether they have achieved poses and their progress in reaching their goals in yoga through YogaPose.

Approach

Our proposed architecture utilizes the Jetson TX2 for local image acquisition and inference, and IBM Cloud services including Object Storage, MongoDB, and GPU enabled Cloud VMs. The high level architecture is outlined in Figure 1.

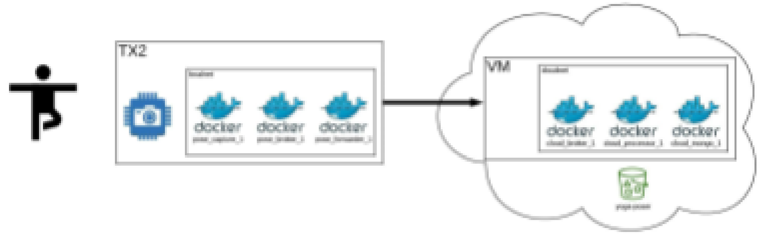


Figure 1. Architecture Diagram

Onboard the TX2, we run two different CNN based Machine Learning models. The first is PoseNet, which was originally proposed by Dan Oved out of Google Creative Labs[8]. The second model is a Yoga Pose Classification model that we have trained from a dataset scraped from Google's Image Download (*refer to Yoga Pose Classification for more information on corpus generation and classification model creation*). In the cloud, we host our MQTT brokers, Cloud Object Storage, and MongoDB.

The high level flow of information involves an instructor sending out a pose to the students, who are then required to strike that pose. A camera peripheral attached to the TX2 captures the human's efforts, and the images are passed into the pose estimation model. The original image, estimated pose, and classified yoga pose are sent back to the cloud for long term storage. The following sections describe each of these steps in greater detail.

PoseNet

As mentioned above, PoseNet is a result of research from Google's Creative Labs. Google's official implementation was in JavaScript with the TensorFlow.js library, however we used port of the model to Python TensorFlow by GitHub user rwightman[11]. Our use case required slight modification of the source code, but the majority of the model conversion and pose extraction logic were drawn directly from the project.

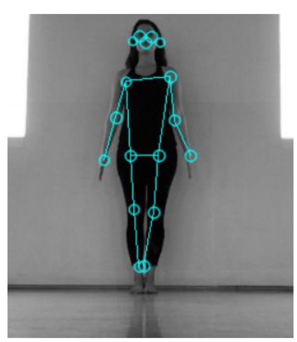
We used a MobileNet backbone architecture for feature extraction. After feature map extraction, the model proposes vectors and confidence values for each body part at each neuron in the feature map. These vectors and confidence values are combined to generate a heatmap of the most likely positions for each body part in the image. User provided thresholds determine which regions of this heatmap are likely body parts, and which are just noise. If the use case requires multiple poses to be extracted, a greedy algorithm is applied to join the different body parts with the instances of human poses. Once these body parts are constructed, we can connect the different body parts to create a wire-frame representing the extracted human pose.

Yoga Pose Classification

We could identify poses in images, but we did not have a dataset. While many premier image datasets are available, like COCO[6], we were unable to find a large enough dataset that focused purely on yoga poses. As a result, we used Google Images Download with chromedriver to search for a variety of different terms associated with each yoga pose, used PoseNet to validate that the images we downloaded contained people performing poses, and then binned these poses based on the search terms used for model development. We requested 50,000 images per search term and downloaded around 10,000 .png and .jpg images tota and generated the dataset using the following approach:

1) Download images for nine yoga poses using a variety of search terms and image types. For instance, our download code requested 50,000 jpgs and .pngs for “mountain” AND “pose”, “mountain” AND “yoga” and Tadasana.
2) We would download about 1200 images per pose, in total generating around 10,000 images. As such, we as a team could not validate that each image contained a yoga pose, so we leveraged PoseNet to do the work for us.
    a) We found that we got the best performance from PoseNet when we converted images to black and white before using PoseNet to identify poses.
    b) Input image examples:


    c) Output image examples:




d) About 60% of the images made it through the PoseNet filter approach. PoseNet was far better at detecting people in standing positions with two feet on the ground below their torso than inverted or crunched poses.

3) We used these 6000+ black and white images with PoseNet skeletons overlayed in color to train and test our model up to a 60% accuracy level on the test set of images. The model itself was a derivation of work shared by Amarchenkova on GitHub.[7]

4) The model was trained was transferred to and deployed on the Tx2.

Cloud Storage

As previously mentioned, while images are captured and classified on the edge device (with immediate results displayed to the student), we also store the raw images and metadata on the cloud for future analysis and to re-train the classification model itself.

While the student views the live results, our application samples one frame each second and encapsulates it, along with its related metadata, into a JSON packet that is sent to a local MQTT queue hosted within a separate container. A python script in a third local container subscribes to that topic and, when a message is received, forwards the image/metadata packet to a separate MQTT broker hosted within a container on a cloud-based virtual machine.

Another cloud-based container monitors that second MQTT topic. When it finds a new message in the queue, it decomposes the payload, reconstructing the captured image to store in a cloud object store bucket and inserting the metadata into mongoDB on a third cloud container.

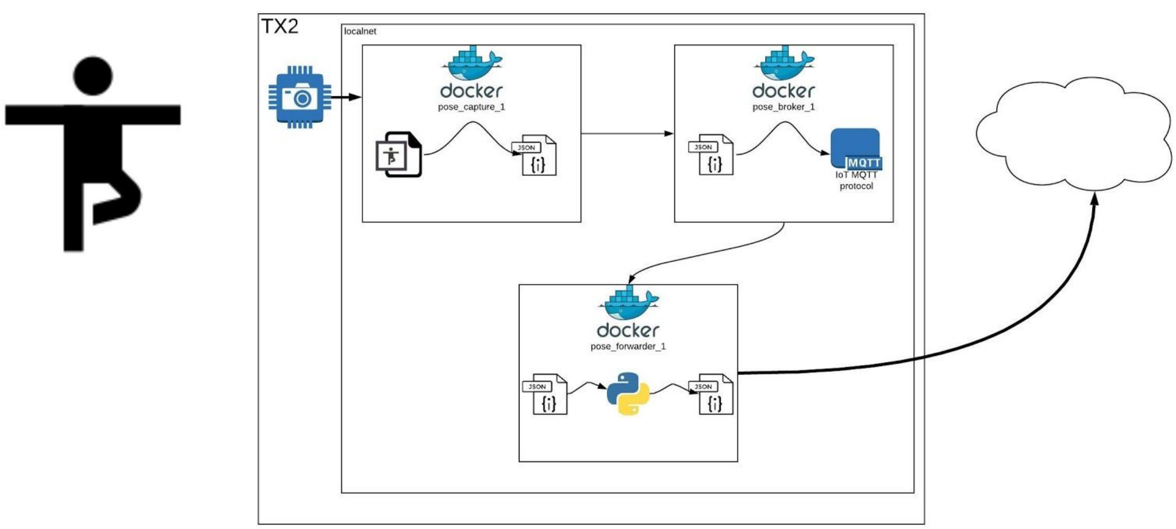


Figure 2: TX2 Infrastructure and TX2 to Cloud Data Transfer

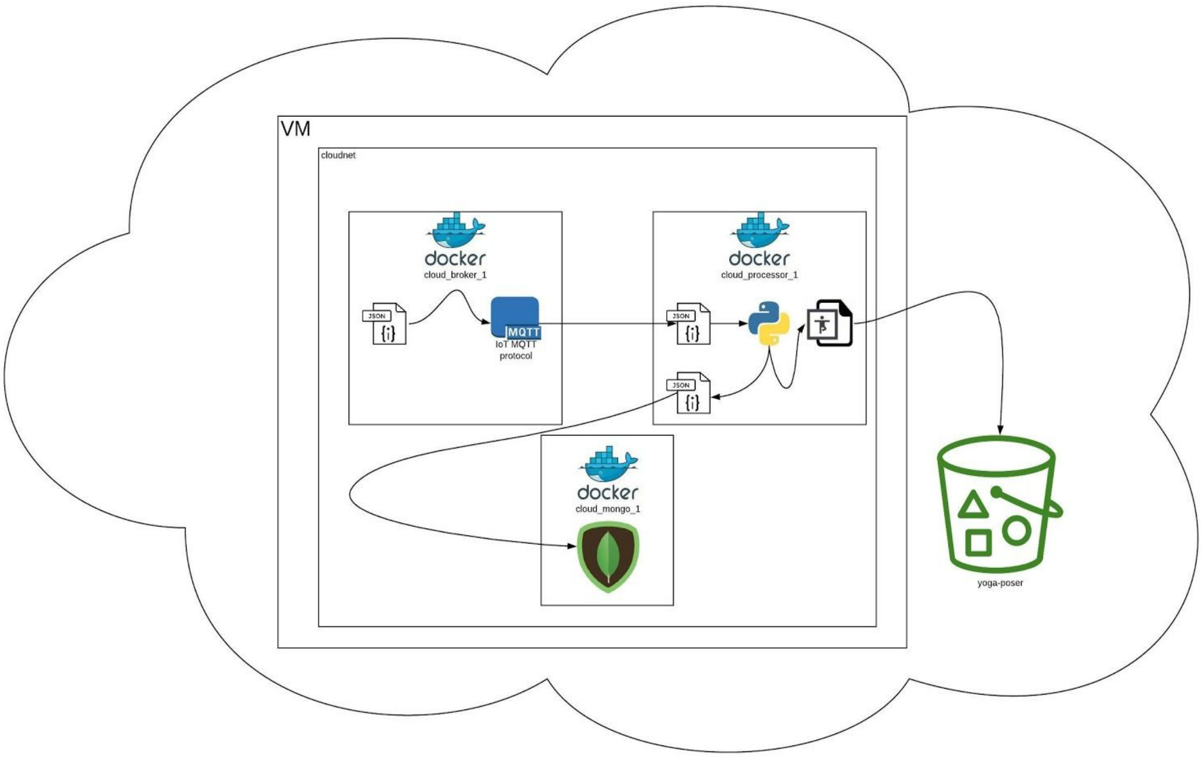


Figure 3: Detailed Breakdown of Cloud Architecture (MQTT Broker, Image Storage, Analytics Storage)

Results

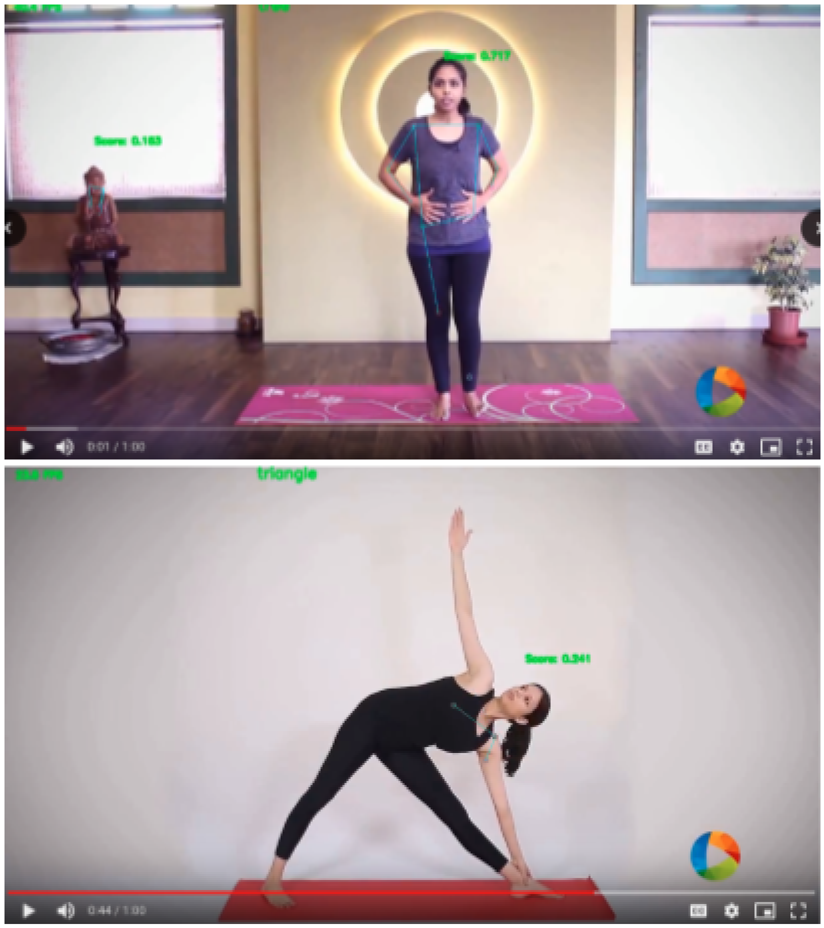


Figure 4: Screenshots of Evaluation Video

We evaluated our model on an instructional video lead by YouTube yogis. Figure 2 displays some frames from that video. When the human was in the upright position, our model typically did quite well at estimating their pose. However, when the human was inverted, the model struggled much more. We believe this is due to a lack of inverted humans in the training data. In order to correct this, we propose gathering more training data of individuals in unusual poses, or inverting the training images as a preprocessing step.

Observing the video, you can also note that the classification model is rather jittery and often incorrect. We believe this is because not all frames have the individual fully in picture, and because of noise in our dataset. With data cleaning, and better filtering of the applied frames, we feel that our classification model's performance would improve even further.

Other Approaches

In a production environment, it would be worthwhile to generate training data specifically for yoga pose classification. It might be interesting to see if a model built with this clean data would require preprocessing from PoseNet, or if it could interpret the images directly. It likely would process crouched and non-standing poses better than the model trained using PoseNet filtered data due to PoseNet's difficulty detecting crouched and non-standing poses. There is also additional opportunity for data collection for the cloud analytics, such as how long the user does yoga per day, the user's self-reported age, weight, etc., so we could update the model based on specific users and track health trends over time. Users may get more detailed feedback if we required the processing to run in the cloud, and the user only interfaced with the edge device. Though it would limit feedback if the user was disconnected from the internet, it might be a useful extension for users with reliable internet connection.

Conclusion

Our approach yields useful, real-time insights in a market where feedback is a prerequisite for improvement. With affordable hardware and optimized models, we are able to extract insights from our consumers similar companies in the industry currently do not have the ability to gather. While this is research is strictly a prototype, it yields a positive outlook on the future of professional in-home athletic instruction.

Github Link: https://github.com/AustinDoolittle/W251FinalProject

Sources Cited

[1] Be Prepared For Race Day. (n.d.). Retrieved from https://www.trainingpeaks.com/.

[2] Buy Apple Watch Series 5. (n.d.). Retrieved from https://www.apple.com/shop/buy-watch/apple-watch.

[3] Cao, Z., Simon, T., Wei, S.-E., & Sheikh, Y. (2017). Realtime Multi-person 2D Pose Estimation Using Part Affinity Fields. 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). doi: 10.1109/cvpr.2017.143

[4] Exercise Bike With Indoor Cycling Classes Streamed Live & On-Demand. (n.d.). Retrieved from https://www.onepeloton.com/.

[5] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2017). ImageNet classification with deep convolutional neural networks. Communications of the ACM, 60(6), 84–90. doi: 10.1145/3065386

[6] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., … Zitnick, C. L. (2014). Microsoft COCO: Common Objects in Context. Computer Vision – ECCV 2014 Lecture Notes in Computer Science, 740–755. doi: 10.1007/978-3-319-10602-1_48

[7] Marchenkova, A. (2019, July 1). Retrieved from https://github.com/amarchenkova/yoga-pose-CNN

[8] TensorFlow. (2018, September 27). Real-time Human Pose Estimation in the Browser with TensorFlow.js. Retrieved from https://medium.com/tensorflow/real-time-human-pose-estimation-in-the-browser-with-tensorflow-js-7dd0bc881cd5.

[9] The World's Most Intelligent Home Gym and Personal Trainer. (n.d.). Retrieved from https://www.tonal.com/.

[10] Who We Are. (n.d.). Retrieved from https://www.fitbit.com/about.

[11] Wightman, R. (2019, July 8). Retrieved from https://github.com/rwightman/posenet-python